

# End to End Fraud Analytics & Predictive Modeling Framework

Using AWS, Databricks, and Machine Learning to Analyze  
and Predict Financial Fraud Patterns

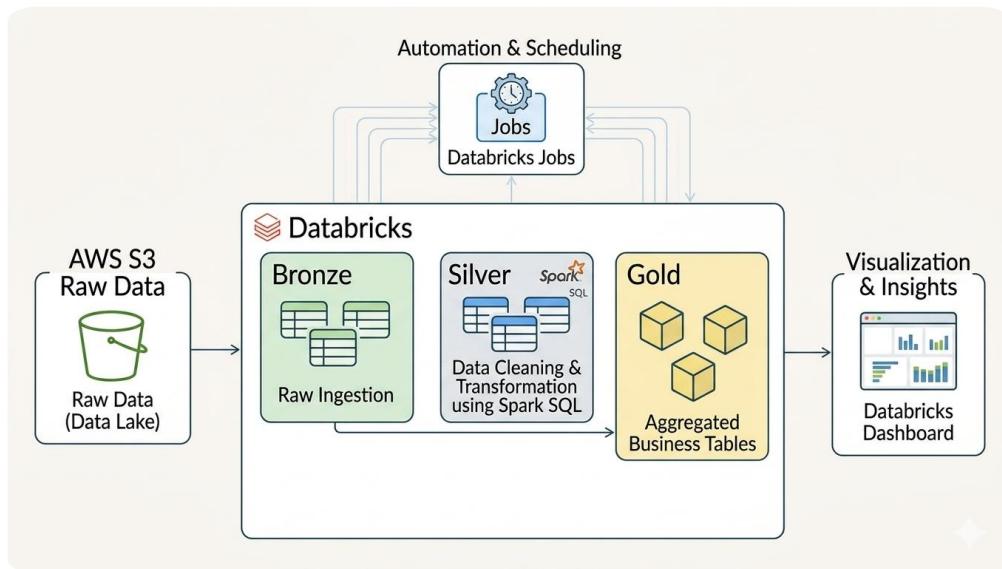




# Background

- Financial institutions increasingly rely on data-driven systems to detect and prevent fraudulent and money laundering activities.
- In this project, a synthetic financial transactions dataset representing a banking context was used to simulate real-world transactional behavior across accounts, customers, and payment channels.

# Architecture overview



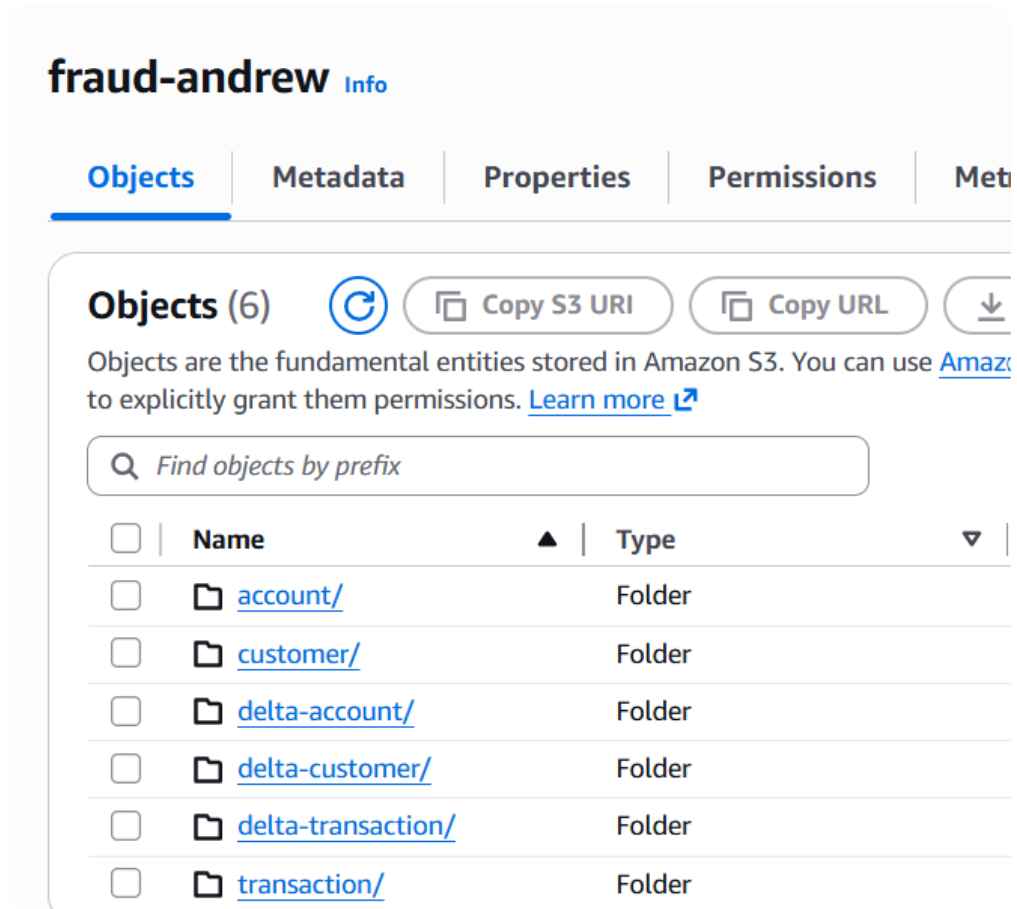
- The data was ingested into **AWS as a data lake** and processed using a **Medallion Architecture in Databricks (Bronze, Silver, and Gold layers)**.

# Problem statement



- Financial fraud and money laundering remain critical risks in modern banking systems, particularly in places where transaction monitoring systems are still evolving or lacking.
- This project aims to analyze and predict fraudulent and suspicious financial behavior using transactional and customer-level data.

# Bronze Layer – Raw Ingestion



Raw fraud data ingested from **AWS s3** into Databricks.

Stored in its original format using **Delta tables**

Ingestion performed using **COPY INTO**.

Metadata (ingestion date) added to track data arrival timelines

# Bronze Layer – Raw Ingestion

▶ Run all (1000) ✓ 1 minute ago (43s) fraud. bronze ▼

```
> CREATE TABLE IF NOT EXISTS bronze.customer_raw(
)
USING DELTA
LOCATION 's3://fraud-andrew/delta-customer';

COPY INTO bronze.customer_raw
FROM 's3://fraud-andrew/customer'
FILEFORMAT = CSV
FORMAT_OPTIONS ('header' = 'true');

ALTER TABLE bronze.customer_raw
ADD COLUMNS (
  IngestionDate TIMESTAMP,
  SourceFile STRING
);

UPDATE bronze.customer_raw
SET
  IngestionDate = current_timestamp(),
  SourceFile = 's3://fraud-andrew/customer'
WHERE IngestionDate IS NULL
```

## Key Activities

- **Landing raw data** from S3 into Bronze Delta tables.
- **Metadata enrichment:** Ingestion timestamp & Data source identifier included

## Outcome

- Final Bronze tables containing raw, ingested data ready for transformation thus ensuring preservation of source data and preparing it for downstream transformation

# Silver layer transformation

Silver layer created to convert raw data into clean, analytics-ready datasets.

## Data Cleaning

- Null values and wrong formats removed
- Invalid values corrected or removed.

## Deduplication

- Ranking by primary key and ordering by date.

```
▶ Run all (1000) ✓ 12:50 PM (46s) fraud.silver New SQL editor: ON ☆ L

--step2:create staging view to clean data
CREATE OR REPLACE VIEW customer_staging AS
SELECT
  CustomerID,
  regexp_replace(CustomerName, '@', '') AS CustomerName,
  initcap(Country) AS Country,
  RiskCategory,
  KYCStatus,
  Occupation,
  coalesce(
    try_to_date(trim(EffectiveDate), 'MM/dd/yyyy'),
    try_to_date(trim(EffectiveDate), 'dd/MM/yyyy'),
    try_to_date(trim(EffectiveDate), 'yyyy-MM-dd')) AS EffectiveDate,
  IngestionDate,
  --flag bad data
CASE
  WHEN coalesce(
```

# Silver layer - modeling & merge logic

```
ON target.TransactionID = source.TransactionID
```

```
WHEN MATCHED THEN UPDATE SET
```

```
target.CustomerSK = source.CustomerSK, --- this needs to be included
target.AccountSK = source.AccountSK, ---this needs to be included
target.TransactionDate = source.TransactionDate,
target.Amount = source.Amount,
target.TransactionType = source.TransactionType,
target.Channel = source.Channel,
target.DestinationCountry = source.DestinationCountry,
target.IsFraud = source.IsFraud,
target.IsLaundered = source.IsLaundered,
target.RiskScore = source.RiskScore
```

```
WHEN NOT MATCHED THEN INSERT (
```

```
TransactionID,
CustomerSK,
AccountSK,
```

## Data Enrichment

- Joining multiple datasets to create unified analytical views.

“Data enriched by joining multiple datasets to create a unified view.”

## Merge Logic

- Up-sert strategy: update existing records, insert new ones.



# Gold layer – curation



This layer was developed to deliver business metrics and aggregated insights.

## Business Questions Addressed

- Which account types are most associated with suspicious (laundering) activity?
- What proportion of transactions are fraudulent vs non-fraudulent?
- Are certain account types more exposed to laundering risk?
- Which country shows higher exposure to suspicious financial activity?
- Are there specific days associated with higher financial risk?

# Gold layer – analysis

```
--average risk score by customer name by day of the week
CREATE VIEW IF NOT EXISTS gold.riskscore_dow AS
SELECT
    silver.customer_dim.CustomerName AS Name,
    date_format(silver.transaction_fact.TransactionDate, 'E') AS DayOfWeek,
    ROUND(AVG(silver.transaction_fact.RiskScore)) AS AvgRiskScore
FROM silver.transaction_fact
JOIN silver.customer_dim ON silver.transaction_fact.CustomerSK = silver.customer_dim.CustomerSK
GROUP BY silver.customer_dim.CustomerName, date_format(silver.transaction_fact.TransactionDate, 'E')
ORDER BY Name, DayOfWeek;
```

Temporary views created to support the computations below;

- SELECT \* FROM gold.account\_type\_launderingrate;
- SELECT \* FROM gold.Uganda\_Fraud;
- SELECT \* FROM gold.luandering\_account;
- SELECT \* FROM gold.Country\_LaunderingRate;
- SELECT \* FROM gold.riskscore\_dow

# hboard



- **Insights;**
  - **Laundrying rate by account** The **Current** account (blue) has the highest rate at 43.16%.
  - **Fraud in Uganda:** Bar chart on the left yellow vs blue indicates a high volume of confirmed fraud cases in Uganda
  - **Channel laundrying vis-à-vis KYC status:** this indicates that pending accounts under mobile channel have the highest laundrying rate
  - **Country fraud volume:** Uganda has higher fraud cases standing at 75% while Kenya is at 25%. As Kenya has no zero fraud cases.
  - **Risk score by day of week(bottom):** shows high-risk individuals & how their risk fluctuates, with some peaking on days like Friday or Monday.

# Fraud Classification Model (Logistic Regression)

File Edit View Run Help Python ☆ Last edit was 3 hours ago

```
##LOGISTICAL MODEL PROCESS FLOW
# step0: Select catalog & schema
# step1: Link the notebook to the experiment (Fraud-Prediction)
# step2: Load derived features table into a Pandas DataFrame
# step3: define independent variable -x and dependent variables -y
# step4: Split training and testing data
# step5: Train the logistic regression model
# step6: define preds and probs
# step7: Compare actual values with model predictions and probabilities.
# step8: Save results for dashboard / SQL
# step9: Evaluate accuracy
# step10: Inspect coefficients
```

- **Objective** - Identify the key factors that increase the likelihood of a transaction being fraudulent.
- **Model used** - Logistic Regression (binary classification: Fraud vs Non-Fraud)
- **Input features;**
  - Transaction count for last 7 days,
  - Average amount for last 30 days,
  - Average amount versus amount transacted,
  - does transaction have foreign destination, customer tenure
- **What the model does** – estimates the probability that a transaction is fraudulent based on historical patterns and customer behavior.
- **Business value**
  - Early detection of suspicious transactions
  - Supports real time fraud prevention
  - Helps reduce financial losses and improve compliance

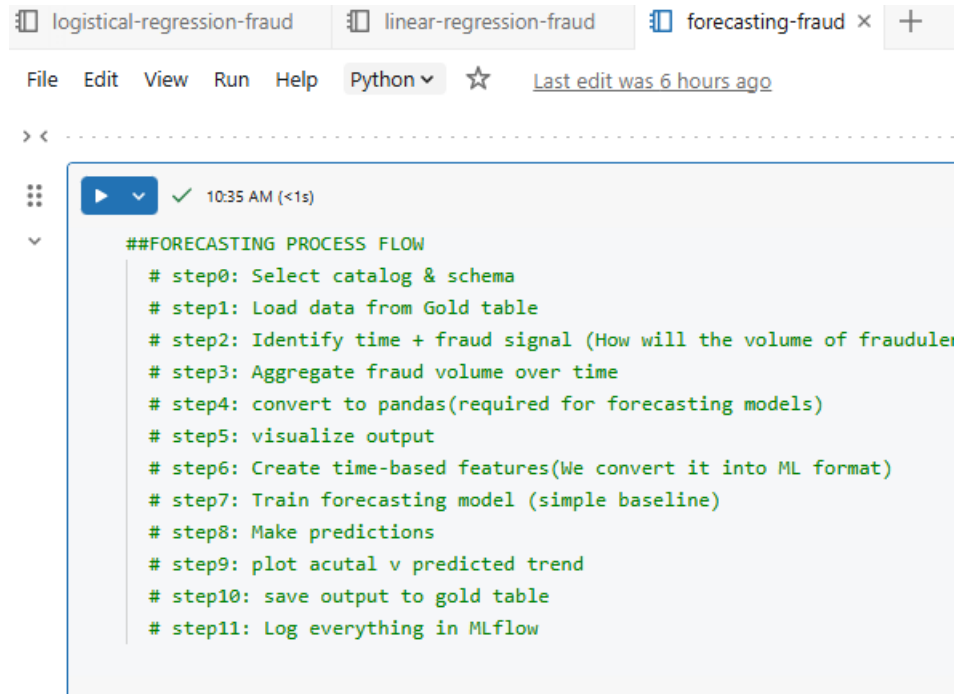
# Risk Score Prediction Model (Linear Regression)

File Edit View Run Help Python ▾ ☆ Last edit was 4 hours ago

```
##LINEAR MODEL PROCESS FLOW
# step0: Select catalog & schema
# step1: import mlflow and link experiment to notebook
# step2: load derived features table into pandas dataframe
# step3: define variables and train the linear regression model
# step4: Evaluate the model and define linear df
# step5: save output for dashboard using SQL
# step6: Interpret (coefficients)
# step7: Log to MLflow
```

- **Objective** - To understand how customer and transaction characteristics influence the overall risk score of a transaction.
- **Model used** - Linear Regression to predict continuous risk score
- **Input features;**
  - Transaction count for last 7 days,
  - Average amount for last 30 days,
  - Average amount versus amount transacted,
  - does transaction have foreign destination, customer tenure
- **What the model does** – it quantifies how each factor contributes to increasing or decreasing transaction risk.
- **Business value;**
  - Improve customer risk profiling
  - Helps banks adjust monitoring thresholds
  - Supports risk-based decision making

# Fraud Volume Forecasting



```
##FORECASTING PROCESS FLOW
# step0: Select catalog & schema
# step1: Load data from Gold table
# step2: Identify time + fraud signal (How will the volume of fraudulent transactions change over time)
# step3: Aggregate fraud volume over time
# step4: convert to pandas(required for forecasting models)
# step5: visualize output
# step6: Create time-based features(We convert it into ML format)
# step7: Train forecasting model (simple baseline)
# step8: Make predictions
# step9: plot actual v predicted trend
# step10: save output to gold table
# step11: Log everything in MLflow
```

**Objective** - To forecast how the volume of fraudulent transactions changes over time.  
Model used - Baseline forecasting using Linear Regression on time index (Day Index)

**Input features;**

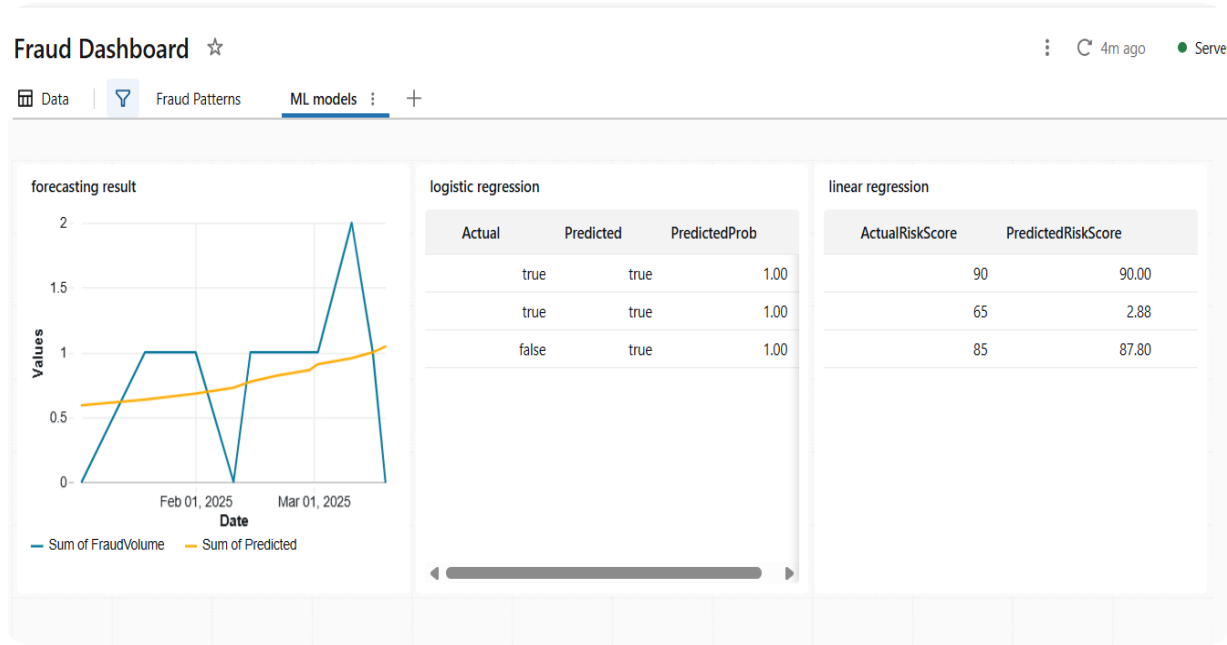
- Date (converted to time index)
- Historical fraud volume per day

**What the model does** - The model learns the overall trend of fraud occurrence over time and projects future values.

**Business value;**

- Helps anticipate fraud spikes
- Supports staffing and monitoring resource planning
- Enables proactive fraud detection strategies

# ML dashboard

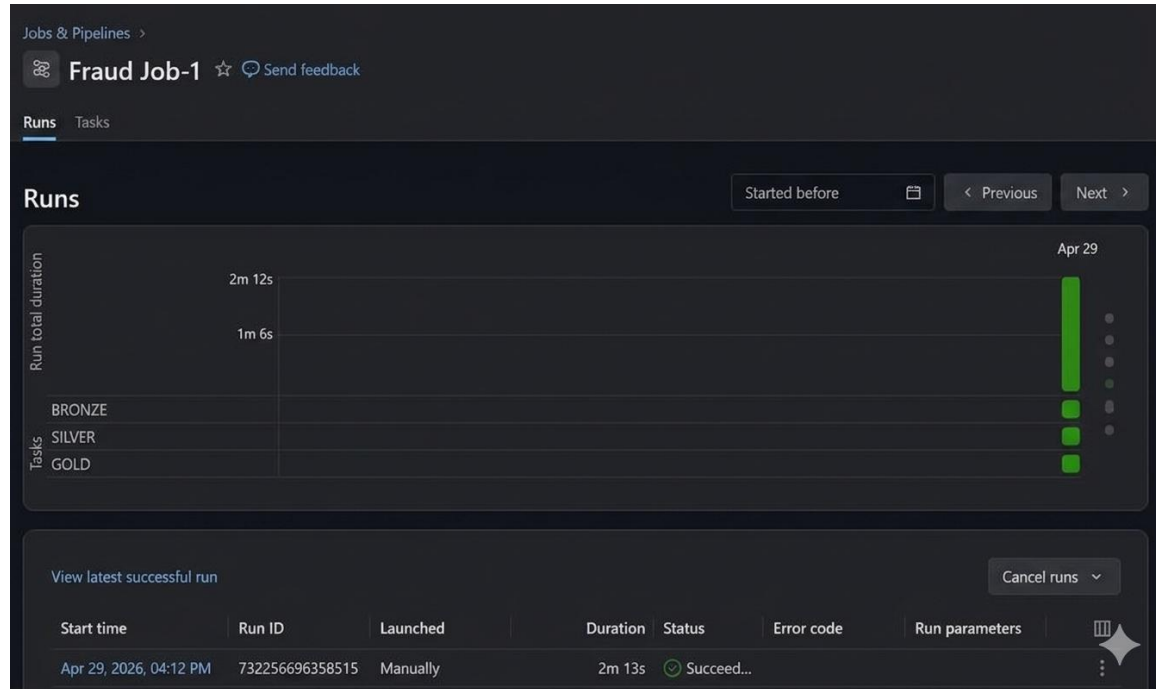


- **Forecasting result:** the Sum of Fraud Volume (blue line) shows extreme volatility with sharp peaks. In contrast, the Sum of Predicted (yellow line) shows a steady, slow upward trend.
  - **Prediction Lag:** The model is not currently capturing the sharp, irregular spikes in actual fraud volume
- **Logistic regression result:** the model is predicting "true" (fraud) but fails on false (no fraud)
- **Linear regression result:** for the second record, there is a big discrepancy: an Actual Risk Score of **65** vs. a Predicted Risk Score of only **2.88**.
  - While the first and third records are very close (90 vs 90 and 85 vs 87.80), the middle record suggests the model is failing to identify risk in certain scenarios, which could lead to "missed" fraud cases.

# Workflow Automation

## Databricks Jobs

- Automated the entire pipeline across Bronze → Silver → Gold.
- This ensured consistent, scheduled processing.







# Final Deliverables

- ✓ End-to-end Delta Lakehouse pipeline
- ✓ Raw bronze datasets for traceability
- ✓ Transformed Silver datasets
- ✓ Business-ready Gold tables
- ✓ Interactive performance dashboard
- ✓ Automated Databricks workflow
- ✓ Fraud Risk Modeling and Forecasting Framework